Final Project - Walmart Data Analysis

What influences sales. We want some inventory decision making so that we can optimize our stocking

Dependent Variable: Weekly Sales Revenue

- strongest business relevance,
- easiest executive storytelling,
- supports all required models,
- easier visualizations,
- cleaner forecasting,
- and usually better model performance.

Analysis Includes: regression models predicting sales, random forest models identifying demand drivers, and time series forecasts for future inventory planning.

In [277…

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy.stats as stats

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

import statsmodels.api as sm
import statsmodels.formula.api as smf
```

In [278…

```
df = pd.read_csv('Walmart.csv')
df.head()
```

Out[278…

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Demand Forecast	Pric
0	1/1/2022	S001	P0001	Groceries	North	231	127	55	135.47	33.5
1	1/1/2022	S001	P0002	Toys	South	204	150	66	144.04	63.0
2	1/1/2022	S001	P0003	Toys	West	102	65	51	74.02	27.9
3	1/1/2022	S001	P0004	Toys	North	469	61	164	62.18	32.7
4	1/1/2022	S001	P0005	Electronics	East	166	14	135	9.26	73.6

Data Cleaning and Prep

Drop columns that were imported but don't contain data.

In [279…
```
df = df.iloc[:, :-2]
df.columns = df.columns.str.strip()
df.drop(columns=['Demand Forecast'], inplace=True)
df.head()
```

Out[279…

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Discour
0	1/1/2022	S001	P0001	Groceries	North	231	127	55	33.50	2
1	1/1/2022	S001	P0002	Toys	South	204	150	66	63.01	2
2	1/1/2022	S001	P0003	Toys	West	102	65	51	27.99	1
3	1/1/2022	S001	P0004	Toys	North	469	61	164	32.72	1
4	1/1/2022	S001	P0005	Electronics	East	166	14	135	73.64	

In [280…
```
# Convert Date
df['Date'] = pd.to_datetime(df['Date'])

# Create date-based features for modeling - added to help with modeling after tryin
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['DayOfWeek'] = df['Date'].dt.dayofweek
```

Check for null values

In [281…
```
df.isnull().values.any()
```

Out[281… `np.False_`

Check for outliers and overall data scan

In [282…
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 73100 entries, 0 to 73099
Data columns (total 17 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Date                73100 non-null  datetime64[us]
 1   Store ID            73100 non-null  str
 2   Product ID          73100 non-null  str
 3   Category            73100 non-null  str
 4   Region              73100 non-null  str
 5   Inventory Level     73100 non-null  int64
 6   Units Sold          73100 non-null  int64
 7   Units Ordered       73100 non-null  int64
 8   Price               73100 non-null  float64
 9   Discount            73100 non-null  int64
 10  Weather Condition   73100 non-null  str
 11  Holiday/Promotion   73100 non-null  int64
 12  Competitor Pricing  73100 non-null  float64
 13  Seasonality         73100 non-null  str
 14  Year                73100 non-null  int32
 15  Month               73100 non-null  int32
 16  DayOfWeek           73100 non-null  int32
dtypes: datetime64[us](1), float64(2), int32(3), int64(5), str(6)
memory usage: 8.6 MB
```

Outlier check outliers based on 1.5 IQR Above or Below

In [283…

```
numeric_df = df.select_dtypes(include=['number'])

Q1 = numeric_df.quantile(0.25)
Q3 = numeric_df.quantile(0.75)
IQR = Q3 - Q1

outliers = (numeric_df < (Q1 - 1.5 * IQR)) | (numeric_df > (Q3 + 1.5 * IQR))
outliers.sum()
```

Out[283…

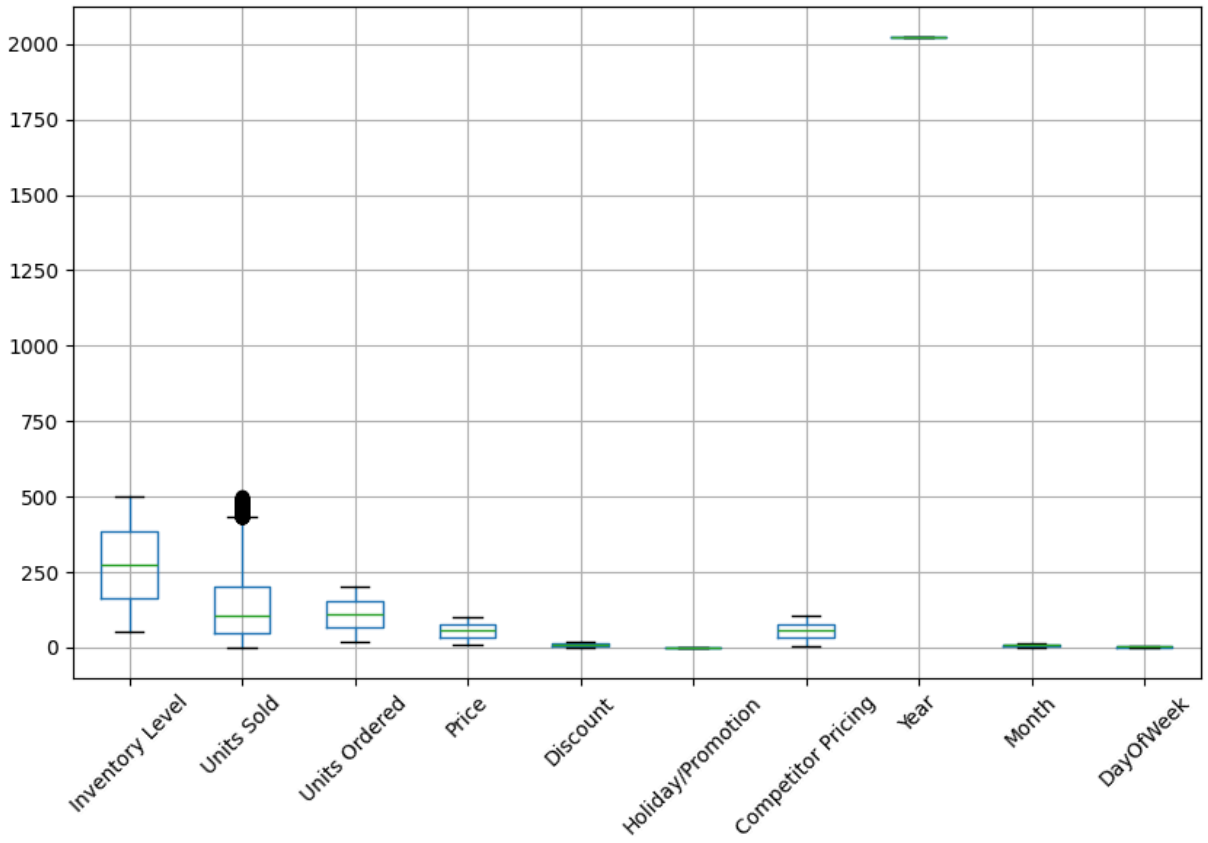
```
Inventory Level         0
Units Sold            715
Units Ordered           0
Price                   0
Discount                0
Holiday/Promotion       0
Competitor Pricing      0
Year                    0
Month                   0
DayOfWeek               0
dtype: int64
```

In [284…

```
from scipy.stats import zscore
import numpy as np

z = np.abs(zscore(df.select_dtypes(include=[np.number])))
outliers = (z > 3)

np.sum(outliers, axis=0)
```

Out[284… array([0, 236, 0, 0, 0, 0, 0, 0, 0, 0])

In [285…
```
import matplotlib.pyplot as plt

df.boxplot(figsize=(10,6))
plt.xticks(rotation=45)
plt.show()
```



In [286…
```
df.columns
```

Out[286…
```
Index(['Date', 'Store ID', 'Product ID', 'Category', 'Region',
       'Inventory Level', 'Units Sold', 'Units Ordered', 'Price', 'Discount',
       'Weather Condition', 'Holiday/Promotion', 'Competitor Pricing',
       'Seasonality', 'Year', 'Month', 'DayOfWeek'],
      dtype='str')
```

In [287…
```
numeric_cols = [
    'Inventory Level',
    'Units Sold',
    # 'Demand Forecast',
    'Holiday/Promotion',
    'Units Ordered',
    'Price',
    'Discount',
    'Competitor Pricing'
]
```

Added log_units_sold to see if the helps with outliers. The data seems valid, I want to see if the log value is beneficial.

In [288…
```
df['log_units_sold'] = np.log1p(df['Units Sold'])
```

Ensure Discount is not a negitive value

In [289…
```
df[(df['Discount'] < 0)]
```

Out[289…

Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Discount	V Co

In [290…
```
import pandas as pd

pd.set_option('display.max_rows', None)

df[outliers.any(axis=1)]
```

Out[290...

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
102	2022-01-02	S001	P0003	Clothing	South	488	464	163	70.99	
138	2022-01-02	S002	P0019	Toys	East	492	479	132	79.11	
554	2022-01-06	S003	P0015	Furniture	North	490	476	53	20.66	
750	2022-01-08	S003	P0011	Furniture	East	482	480	68	74.14	
1512	2022-01-16	S001	P0013	Furniture	South	482	482	62	62.11	
1948	2022-01-20	S003	P0009	Toys	South	495	469	165	27.14	
1949	2022-01-20	S003	P0010	Groceries	North	482	471	144	10.73	
2023	2022-01-21	S002	P0004	Electronics	East	488	476	37	32.05	
2151	2022-01-22	S003	P0012	Groceries	North	484	482	141	56.00	
2306	2022-01-24	S001	P0007	Electronics	East	466	466	133	86.56	
2832	2022-01-29	S002	P0013	Furniture	East	486	466	21	97.82	
2927	2022-01-30	S002	P0008	Electronics	North	490	473	103	23.11	
3429	2022-02-04	S002	P0010	Toys	South	497	476	155	41.08	
3526	2022-02-05	S002	P0007	Clothing	East	492	468	44	95.94	
3970	2022-02-09	S004	P0011	Furniture	West	480	478	140	72.92	
3996	2022-02-09	S005	P0017	Toys	West	497	484	91	49.61	
4005	2022-02-10	S001	P0006	Clothing	East	491	489	138	71.79	
4291	2022-02-12	S005	P0012	Electronics	North	498	484	158	34.50	
4452	2022-02-14	S003	P0013	Toys	East	499	476	177	70.67	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
5039	2022-02-20	S002	P0020	Toys	South	469	468	147	33.56	
5317	2022-02-23	S001	P0018	Clothing	North	497	485	102	40.82	
5546	2022-02-25	S003	P0007	Clothing	East	476	471	21	10.03	
5666	2022-02-26	S004	P0007	Electronics	South	497	468	86	15.09	
5669	2022-02-26	S004	P0010	Furniture	South	494	484	27	53.60	
6280	2022-03-04	S005	P0001	Groceries	North	493	468	156	20.58	
6411	2022-03-06	S001	P0012	Groceries	South	490	471	54	53.03	
6475	2022-03-06	S004	P0016	Groceries	West	469	465	113	19.63	
6563	2022-03-07	S004	P0004	Furniture	West	483	465	118	30.40	
6606	2022-03-08	S001	P0007	Groceries	West	480	471	182	45.94	
6669	2022-03-08	S004	P0010	Toys	East	477	474	109	95.70	
6769	2022-03-09	S004	P0010	Furniture	South	499	483	72	80.22	
6773	2022-03-09	S004	P0014	Electronics	North	490	481	98	34.40	
8004	2022-03-22	S001	P0005	Groceries	North	487	476	151	26.10	
8766	2022-03-29	S004	P0007	Groceries	North	495	491	97	12.19	
8928	2022-03-31	S002	P0009	Electronics	South	493	467	174	28.13	
9033	2022-04-01	S002	P0014	Toys	North	490	477	64	12.18	
9140	2022-04-02	S003	P0001	Groceries	South	493	464	146	73.15	
9154	2022-04-02	S003	P0015	Furniture	West	500	475	119	32.84	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
9389	2022-04-04	S005	P0010	Groceries	East	495	468	190	72.06	
9486	2022-04-05	S005	P0007	Clothing	West	468	467	94	30.04	
10035	2022-04-11	S002	P0016	Toys	North	482	476	86	13.65	
10866	2022-04-19	S004	P0007	Groceries	West	475	473	120	85.77	
11016	2022-04-21	S001	P0017	Electronics	East	497	491	72	59.72	
11483	2022-04-25	S005	P0004	Toys	East	481	471	30	33.81	
11521	2022-04-26	S002	P0002	Furniture	North	496	470	124	40.15	
11720	2022-04-28	S002	P0001	Toys	South	480	474	158	29.32	
11895	2022-04-29	S005	P0016	Toys	East	488	475	101	39.48	
11964	2022-04-30	S004	P0005	Furniture	North	493	476	83	72.49	
11966	2022-04-30	S004	P0007	Clothing	West	492	467	87	57.55	
12066	2022-05-01	S004	P0007	Groceries	West	481	466	100	73.00	
12555	2022-05-06	S003	P0016	Electronics	East	479	466	108	81.13	
12746	2022-05-08	S003	P0007	Toys	East	469	469	197	41.39	
13320	2022-05-14	S002	P0001	Groceries	East	484	473	86	94.11	
13453	2022-05-15	S003	P0014	Electronics	North	481	471	126	47.60	
14213	2022-05-23	S001	P0014	Groceries	North	492	485	162	10.10	
14326	2022-05-24	S002	P0007	Groceries	South	500	477	22	90.79	
15064	2022-05-31	S004	P0005	Clothing	North	486	468	115	43.65	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
15647	2022-06-06	S003	P0008	Electronics	East	479	470	66	19.89	
15744	2022-06-07	S003	P0005	Toys	South	499	478	196	10.50	
15855	2022-06-08	S003	P0016	Clothing	South	500	478	79	19.46	
16085	2022-06-10	S005	P0006	Clothing	South	494	471	118	17.93	
16448	2022-06-14	S003	P0009	Toys	East	499	477	155	53.35	
16529	2022-06-15	S002	P0010	Furniture	South	499	465	103	97.68	
16763	2022-06-17	S004	P0004	Electronics	South	490	489	159	38.60	
17787	2022-06-27	S005	P0008	Furniture	North	494	473	31	53.42	
17952	2022-06-29	S003	P0013	Furniture	North	499	467	88	57.91	
18892	2022-07-08	S005	P0013	Groceries	West	495	480	52	18.66	
19111	2022-07-11	S001	P0012	Toys	West	500	496	101	72.31	
19380	2022-07-13	S005	P0001	Electronics	South	481	470	98	85.65	
19399	2022-07-13	S005	P0020	Clothing	West	490	484	81	21.26	
19550	2022-07-15	S003	P0011	Toys	South	488	484	74	81.40	
19935	2022-07-19	S002	P0016	Electronics	East	495	492	92	30.95	
19977	2022-07-19	S004	P0018	Furniture	West	498	476	59	48.23	
20219	2022-07-22	S001	P0020	Electronics	North	497	473	43	12.94	
20555	2022-07-25	S003	P0016	Groceries	West	493	485	66	76.95	
21522	2022-08-04	S002	P0003	Electronics	South	488	479	28	87.21	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
21598	2022-08-04	S005	P0019	Toys	North	480	473	25	48.51	
21692	2022-08-05	S005	P0013	Toys	East	483	469	167	67.55	
21755	2022-08-06	S003	P0016	Clothing	South	489	484	66	64.40	
21901	2022-08-08	S001	P0002	Clothing	North	496	486	24	10.73	
22031	2022-08-09	S002	P0012	Groceries	North	478	473	51	43.84	
22286	2022-08-11	S005	P0007	Electronics	West	482	468	80	61.01	
22373	2022-08-12	S004	P0014	Electronics	North	495	481	83	71.96	
22803	2022-08-17	S001	P0004	Clothing	West	495	475	39	71.59	
23667	2022-08-25	S004	P0008	Groceries	North	490	469	64	13.59	
23822	2022-08-27	S002	P0003	Clothing	East	499	465	63	10.85	
23856	2022-08-27	S003	P0017	Toys	North	481	470	130	33.35	
23969	2022-08-28	S004	P0010	Toys	North	493	476	77	19.97	
24397	2022-09-01	S005	P0018	Groceries	West	490	475	155	35.61	
24417	2022-09-02	S001	P0018	Electronics	West	493	473	48	10.32	
24918	2022-09-07	S001	P0019	Clothing	North	484	471	52	64.22	
25507	2022-09-13	S001	P0008	Electronics	West	492	485	39	40.43	
25588	2022-09-13	S005	P0009	Clothing	West	479	468	146	85.22	
25695	2022-09-14	S005	P0016	Furniture	East	487	469	103	78.11	
26018	2022-09-18	S001	P0019	Groceries	West	481	475	122	19.69	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
26029	2022-09-18	S002	P0010	Toys	West	488	488	61	14.84	
26031	2022-09-18	S002	P0012	Groceries	South	472	466	54	14.20	
26466	2022-09-22	S004	P0007	Groceries	South	480	464	30	92.68	
27368	2022-10-01	S004	P0009	Clothing	North	497	483	84	94.36	
27607	2022-10-04	S001	P0008	Electronics	North	499	490	175	53.88	
27986	2022-10-07	S005	P0007	Electronics	North	497	465	155	55.39	
28614	2022-10-14	S001	P0015	Groceries	East	496	471	87	38.75	
29698	2022-10-24	S005	P0019	Groceries	East	471	467	185	42.61	
30104	2022-10-29	S001	P0005	Clothing	North	498	477	59	31.64	
31502	2022-11-12	S001	P0003	Groceries	North	500	479	101	63.41	
31610	2022-11-13	S001	P0011	Furniture	West	491	476	89	35.85	
31653	2022-11-13	S003	P0014	Clothing	West	481	475	192	36.41	
32084	2022-11-17	S005	P0005	Clothing	West	494	465	77	76.37	
32384	2022-11-20	S005	P0005	Electronics	East	476	468	67	68.59	
32676	2022-11-23	S004	P0017	Electronics	East	486	475	168	92.13	
32852	2022-11-25	S003	P0013	Toys	West	496	486	176	59.94	
32926	2022-11-26	S002	P0007	Electronics	South	488	478	48	83.85	
33018	2022-11-27	S001	P0019	Furniture	South	470	469	62	85.90	
33766	2022-12-04	S004	P0007	Groceries	East	494	472	153	33.61	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
34538	2022-12-12	S002	P0019	Furniture	North	473	473	138	78.05	
35160	2022-12-18	S004	P0001	Furniture	North	486	485	163	88.93	
35557	2022-12-22	S003	P0018	Electronics	West	499	465	130	66.98	
35688	2022-12-23	S005	P0009	Groceries	North	496	491	196	39.52	
35765	2022-12-24	S004	P0006	Clothing	North	498	480	192	16.14	
35818	2022-12-25	S001	P0019	Groceries	South	474	467	129	34.22	
35946	2022-12-26	S003	P0007	Toys	North	497	485	116	89.05	
36228	2022-12-29	S002	P0009	Clothing	West	479	476	138	81.63	
36571	2023-01-01	S004	P0012	Groceries	East	497	493	25	55.09	
36643	2023-01-02	S003	P0004	Furniture	South	483	467	53	11.95	
36859	2023-01-04	S003	P0020	Groceries	East	485	467	184	88.71	
36953	2023-01-05	S003	P0014	Furniture	North	484	469	112	16.21	
37149	2023-01-07	S003	P0010	Electronics	South	500	487	96	37.29	
37883	2023-01-14	S005	P0004	Furniture	East	496	478	82	69.91	
37924	2023-01-15	S002	P0005	Clothing	East	493	473	167	94.20	
37970	2023-01-15	S004	P0011	Electronics	South	493	478	123	31.41	
38024	2023-01-16	S002	P0005	Furniture	East	488	488	169	14.45	
38128	2023-01-17	S002	P0009	Clothing	East	490	473	24	55.60	
39256	2023-01-28	S003	P0017	Toys	West	498	495	44	67.36	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
39629	2023-02-01	S002	P0010	Groceries	North	491	467	101	96.40	
40287	2023-02-07	S005	P0008	Groceries	East	495	494	56	88.80	
40339	2023-02-08	S002	P0020	Furniture	East	495	479	165	81.92	
40374	2023-02-08	S004	P0015	Toys	South	473	466	153	34.09	
40595	2023-02-10	S005	P0016	Groceries	East	466	464	147	52.42	
40749	2023-02-12	S003	P0010	Furniture	South	489	472	177	76.34	
40787	2023-02-12	S005	P0008	Electronics	North	493	477	134	23.36	
41226	2023-02-17	S002	P0007	Toys	East	484	469	115	49.86	
41506	2023-02-20	S001	P0007	Electronics	West	478	466	163	86.41	
42822	2023-03-05	S002	P0003	Furniture	East	494	485	146	38.16	
42938	2023-03-06	S002	P0019	Furniture	South	494	473	75	84.48	
43949	2023-03-16	S003	P0010	Electronics	East	482	472	185	60.96	
44087	2023-03-17	S005	P0008	Groceries	East	497	488	87	66.05	
44954	2023-03-26	S003	P0015	Furniture	North	482	474	24	46.34	
45136	2023-03-28	S002	P0017	Clothing	North	488	478	112	49.35	
45155	2023-03-28	S003	P0016	Groceries	North	490	472	127	40.20	
45683	2023-04-02	S005	P0004	Toys	South	494	488	25	61.87	
45839	2023-04-04	S002	P0020	Electronics	North	491	478	176	44.32	
46671	2023-04-12	S004	P0012	Furniture	West	483	474	105	60.78	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
46799	2023-04-13	S005	P0020	Electronics	South	491	467	20	53.92	
46850	2023-04-14	S003	P0011	Furniture	North	498	475	56	73.13	
46883	2023-04-14	S005	P0004	Toys	East	490	472	122	92.40	
47313	2023-04-19	S001	P0014	Electronics	West	490	480	65	42.86	
47597	2023-04-21	S005	P0018	Toys	South	494	487	98	96.80	
48187	2023-04-27	S005	P0008	Groceries	East	500	488	100	18.45	
48283	2023-04-28	S005	P0004	Electronics	West	466	465	39	17.28	
48697	2023-05-02	S005	P0018	Furniture	East	481	481	191	63.67	
48945	2023-05-05	S003	P0006	Toys	West	495	467	161	57.61	
49146	2023-05-07	S003	P0007	Furniture	South	486	479	97	57.16	
49150	2023-05-07	S003	P0011	Electronics	East	490	472	195	80.57	
49358	2023-05-09	S003	P0019	Clothing	East	492	466	173	69.63	
49541	2023-05-11	S003	P0002	Toys	South	489	478	165	28.07	
49567	2023-05-11	S004	P0008	Electronics	North	465	465	118	17.70	
49574	2023-05-11	S004	P0015	Furniture	East	482	482	104	25.20	
49860	2023-05-14	S004	P0001	Furniture	East	476	470	27	36.95	
50140	2023-05-17	S003	P0001	Clothing	East	495	465	25	37.80	
50716	2023-05-23	S001	P0017	Toys	South	500	480	130	14.22	
51313	2023-05-29	S001	P0014	Clothing	North	470	465	80	25.58	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
51347	2023-05-29	S003	P0008	Toys	East	474	465	39	48.43	
51362	2023-05-29	S004	P0003	Furniture	South	481	470	77	65.92	
51821	2023-06-03	S002	P0002	Electronics	South	497	484	197	49.12	
52014	2023-06-05	S001	P0015	Groceries	South	485	466	98	69.53	
52176	2023-06-06	S004	P0017	Electronics	South	496	473	161	49.54	
52409	2023-06-09	S001	P0010	Clothing	East	495	467	104	12.55	
52730	2023-06-12	S002	P0011	Electronics	North	486	467	80	41.69	
52931	2023-06-14	S002	P0012	Electronics	North	490	483	57	91.57	
53040	2023-06-15	S003	P0001	Furniture	South	472	469	50	69.52	
53157	2023-06-16	S003	P0018	Furniture	East	499	499	84	19.95	
53404	2023-06-19	S001	P0005	Furniture	West	488	480	186	56.92	
53869	2023-06-23	S004	P0010	Furniture	East	477	469	158	72.66	
54262	2023-06-27	S004	P0003	Electronics	North	497	472	108	80.27	
54282	2023-06-27	S005	P0003	Clothing	South	474	471	56	21.78	
54839	2023-07-03	S002	P0020	Furniture	North	476	473	25	88.91	
54951	2023-07-04	S003	P0012	Furniture	West	488	473	169	91.62	
55161	2023-07-06	S004	P0002	Groceries	East	496	479	178	46.58	
55412	2023-07-09	S001	P0013	Groceries	South	485	464	102	59.78	
55795	2023-07-12	S005	P0016	Furniture	East	498	480	68	36.03	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
55802	2023-07-13	S001	P0003	Electronics	East	493	484	87	22.37	
56225	2023-07-17	S002	P0006	Furniture	South	491	487	118	30.31	
56491	2023-07-19	S005	P0012	Toys	East	474	474	170	51.28	
56659	2023-07-21	S003	P0020	Groceries	West	491	478	55	97.93	
57571	2023-07-30	S004	P0012	Electronics	North	485	471	184	69.77	
57586	2023-07-30	S005	P0007	Groceries	East	489	472	102	22.25	
57789	2023-08-01	S005	P0010	Groceries	West	488	465	24	89.16	
58021	2023-08-04	S002	P0002	Groceries	South	495	468	189	66.74	
58578	2023-08-09	S004	P0019	Clothing	North	477	475	195	77.44	
58945	2023-08-13	S003	P0006	Electronics	North	484	484	161	75.53	
59241	2023-08-16	S003	P0002	Furniture	East	497	483	108	99.09	
59632	2023-08-20	S002	P0013	Toys	North	500	479	92	40.12	
60620	2023-08-30	S002	P0001	Furniture	North	487	467	84	96.41	
61560	2023-09-08	S004	P0001	Toys	East	481	479	80	73.02	
61648	2023-09-09	S003	P0009	Groceries	South	499	488	116	88.48	
63879	2023-10-01	S004	P0020	Furniture	East	500	479	150	50.48	
64010	2023-10-03	S001	P0011	Electronics	South	489	470	38	41.24	
64313	2023-10-06	S001	P0014	Clothing	South	497	478	158	14.67	
64399	2023-10-06	S005	P0020	Groceries	South	489	482	161	59.67	

	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
64582	2023-10-08	S005	P0003	Electronics	East	487	467	114	27.07	
64827	2023-10-11	S002	P0008	Clothing	East	469	466	180	84.49	
65410	2023-10-17	S001	P0011	Clothing	West	498	471	41	93.60	
65451	2023-10-17	S003	P0012	Furniture	South	488	465	76	81.05	
65544	2023-10-18	S003	P0005	Groceries	West	484	469	157	26.17	
65568	2023-10-18	S004	P0009	Clothing	North	475	474	173	43.00	
66003	2023-10-23	S001	P0004	Electronics	South	477	470	92	50.08	
66015	2023-10-23	S001	P0016	Groceries	South	497	465	130	22.17	
66259	2023-10-25	S003	P0020	Furniture	East	492	472	86	28.50	
66298	2023-10-25	S005	P0019	Groceries	South	478	474	168	58.51	
66905	2023-11-01	S001	P0006	Groceries	North	488	479	119	13.77	
67358	2023-11-05	S003	P0019	Furniture	North	474	473	93	65.25	
67717	2023-11-09	S001	P0018	Toys	North	500	479	183	61.20	
67998	2023-11-11	S005	P0019	Groceries	West	484	470	115	92.83	
68832	2023-11-20	S002	P0013	Groceries	South	472	466	22	91.39	
69565	2023-11-27	S004	P0006	Electronics	South	486	474	96	12.51	
69658	2023-11-28	S003	P0019	Clothing	North	478	469	24	36.87	
70082	2023-12-02	S005	P0003	Toys	South	480	474	49	66.77	
70372	2023-12-05	S004	P0013	Clothing	West	481	469	118	99.77	

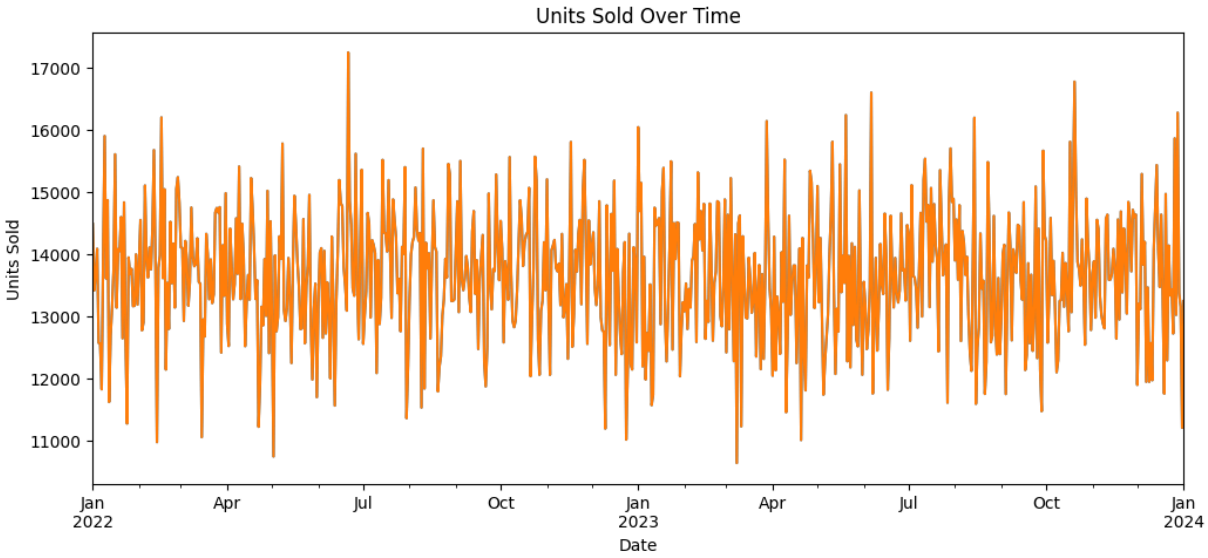
	Date	Store ID	Product ID	Category	Region	Inventory Level	Units Sold	Units Ordered	Price	Disco
70653	2023-12-08	S003	P0014	Electronics	East	497	481	27	18.29	
70789	2023-12-09	S005	P0010	Furniture	South	489	477	111	25.28	
71292	2023-12-14	S005	P0013	Groceries	East	498	494	37	80.53	
71557	2023-12-17	S003	P0018	Furniture	North	489	485	81	51.73	
71558	2023-12-17	S003	P0019	Clothing	South	479	473	167	11.35	
72485	2023-12-26	S005	P0006	Groceries	West	498	482	33	34.79	
72884	2023-12-30	S005	P0005	Furniture	East	498	472	169	95.79	
73060	2024-01-01	S004	P0001	Electronics	East	497	496	137	42.34	

Convert the date column into a time series

In [291…
```
df['Date'] = pd.to_datetime(df['Date'])

df.groupby('Date')['Units Sold'].sum().plot(figsize=(12,5))
```

Out[291…
```
<Axes: xlabel='Date'>
```

In [292…
```
import matplotlib.pyplot as plt

df.groupby('Date')['Units Sold'].sum().plot(figsize=(12,5))
plt.title("Units Sold Over Time")
plt.ylabel("Units Sold")
plt.xlabel("Date")
plt.show()
```



Set Dependent and Independent Variables

In [293…
```
y = df['Units Sold']

X = df.drop(columns=['Units Sold', 'Date'])
```

Fix Residuals

- found issues in the residuals - so came back to add this squared method to fix it.

Residual diagnostics indicated nonlinearity in the model. To address this, polynomial and interaction terms (e.g., price^2 and price × discount) were introduced. This improved model fit and reduced systematic patterns in residuals."

In [294…
```
X = df.drop(columns=['Units Sold', 'Date'])
X = pd.get_dummies(X, drop_first=True)
```

Convert Categorical Values

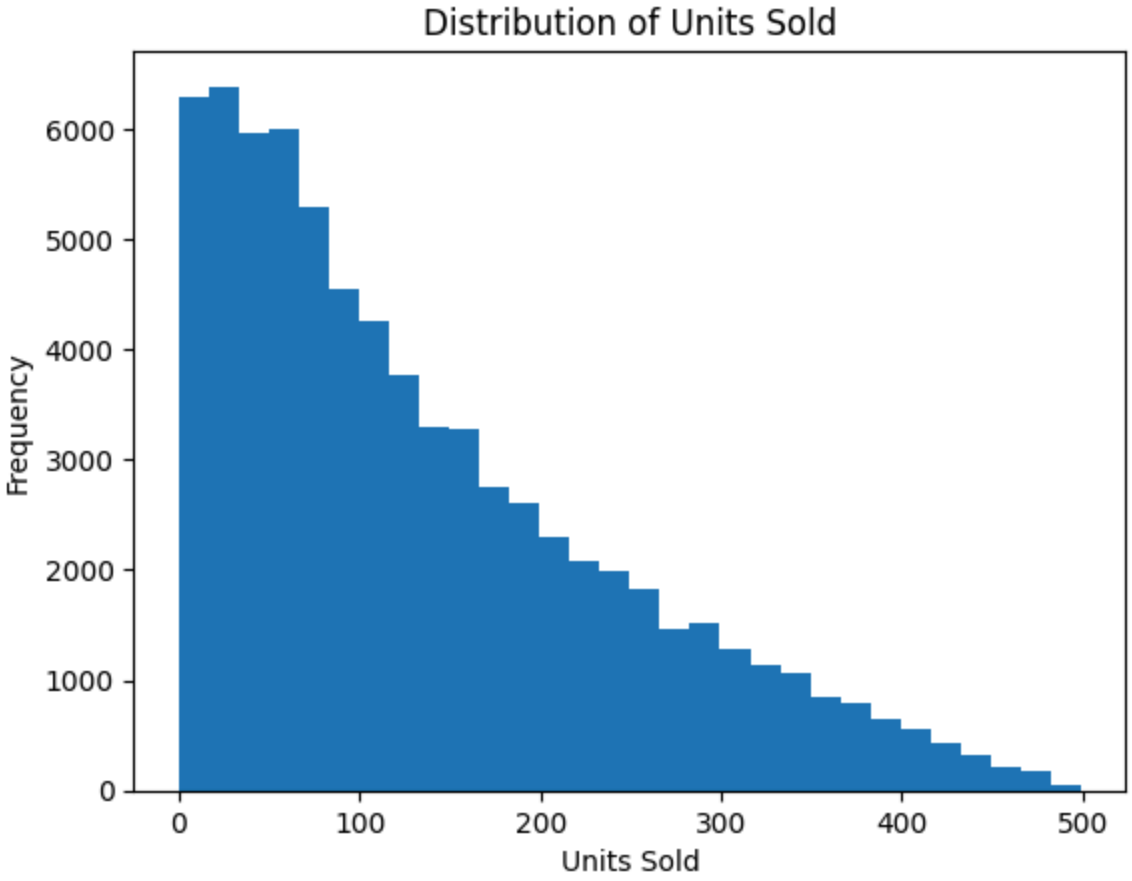
- Get Dummies

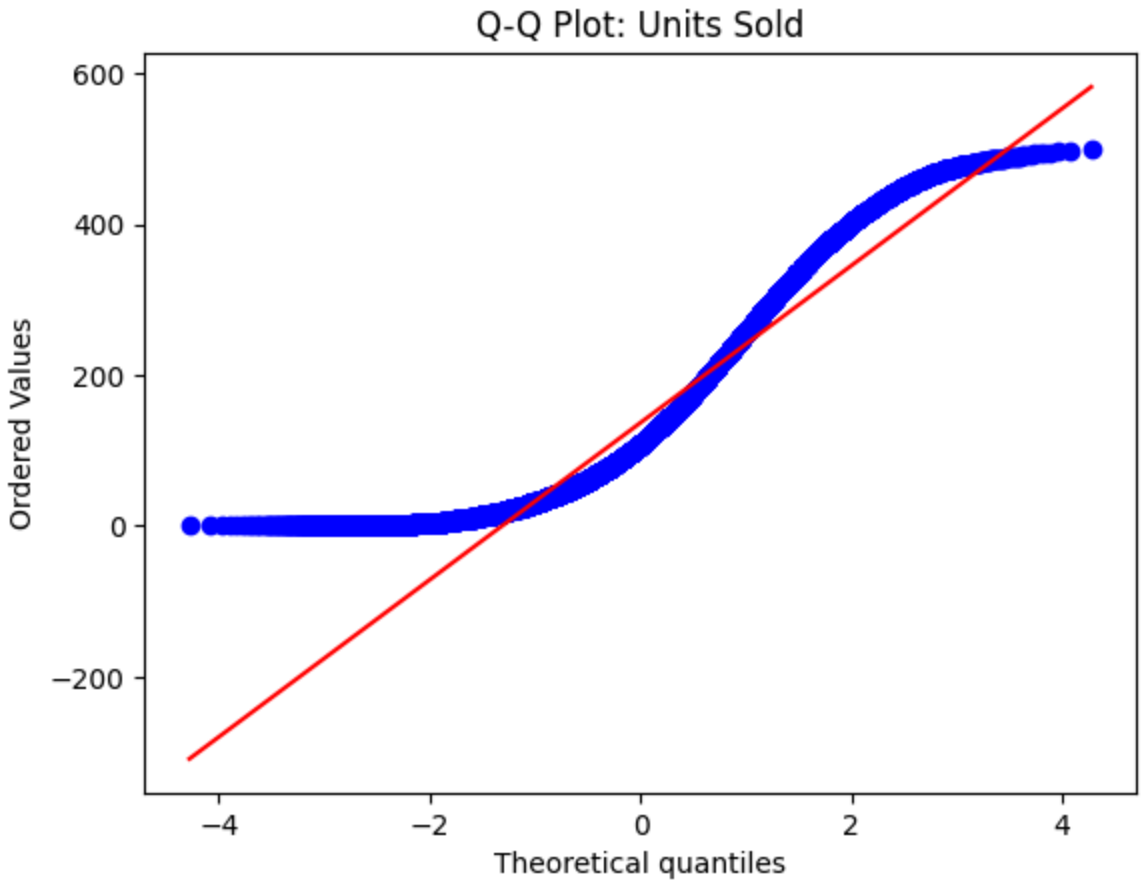
In [295…
```
X = pd.get_dummies(X, drop_first=True)

X = X.astype(float)
```

Check for normality

In [296…
```
plt.hist(y, bins=30)
plt.title("Distribution of Units Sold")
```

```
plt.xlabel("Units Sold")
plt.ylabel("Frequency")
plt.show()

stats.probplot(y, dist="norm", plot=plt)
plt.title("Q-Q Plot: Units Sold")
plt.show()

print("Skewness of Units Sold:", y.skew())
```






```
Skewness of Units Sold: 0.9053326253493985
```

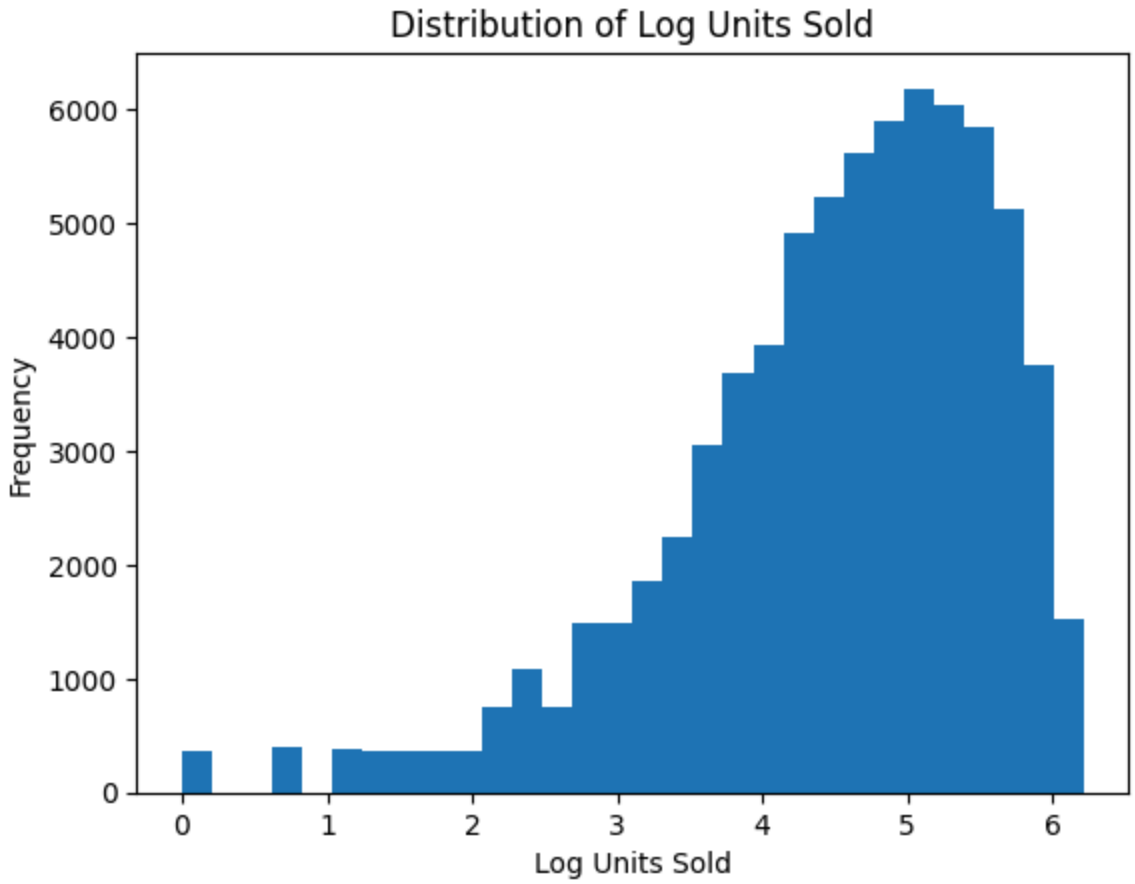
**Log transform to help with normality issue - not great but good enouth. **

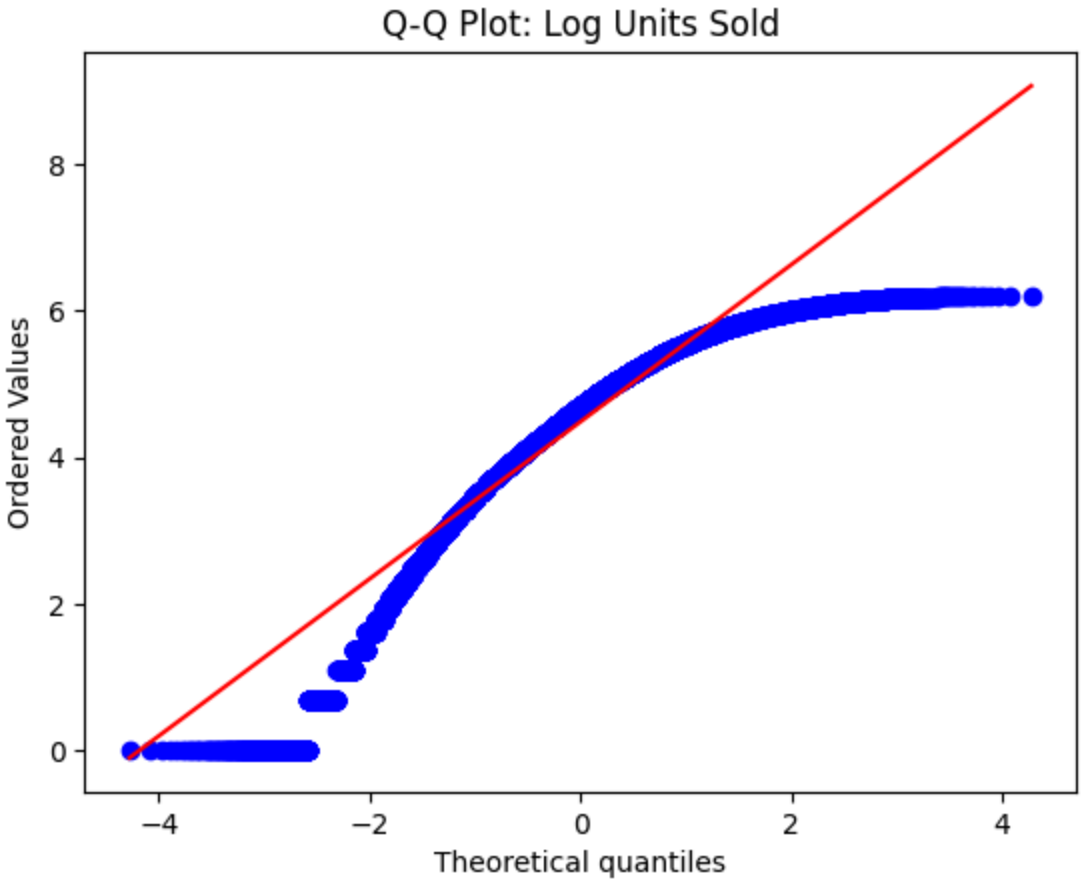
Q-Q Plot That curve shape (S-shape)... not normal, the flat lower tail is due to a lots of zeros and will need to check residuals closely.

Histogram Right-skewed is better, but not great. It is not symatrically or bell-shaped. There is lot of zeros or very small values.

In [297…

```
df['log_units_sold'] = np.log1p(df['Units Sold'])

y_log = df['log_units_sold']

plt.hist(y_log, bins=30)
plt.title("Distribution of Log Units Sold")
plt.xlabel("Log Units Sold")
plt.ylabel("Frequency")
plt.show()

stats.probplot(y_log, dist="norm", plot=plt)
plt.title("Q-Q Plot: Log Units Sold")
plt.show()

print("Skewness of Log Units Sold:", y_log.skew())
```

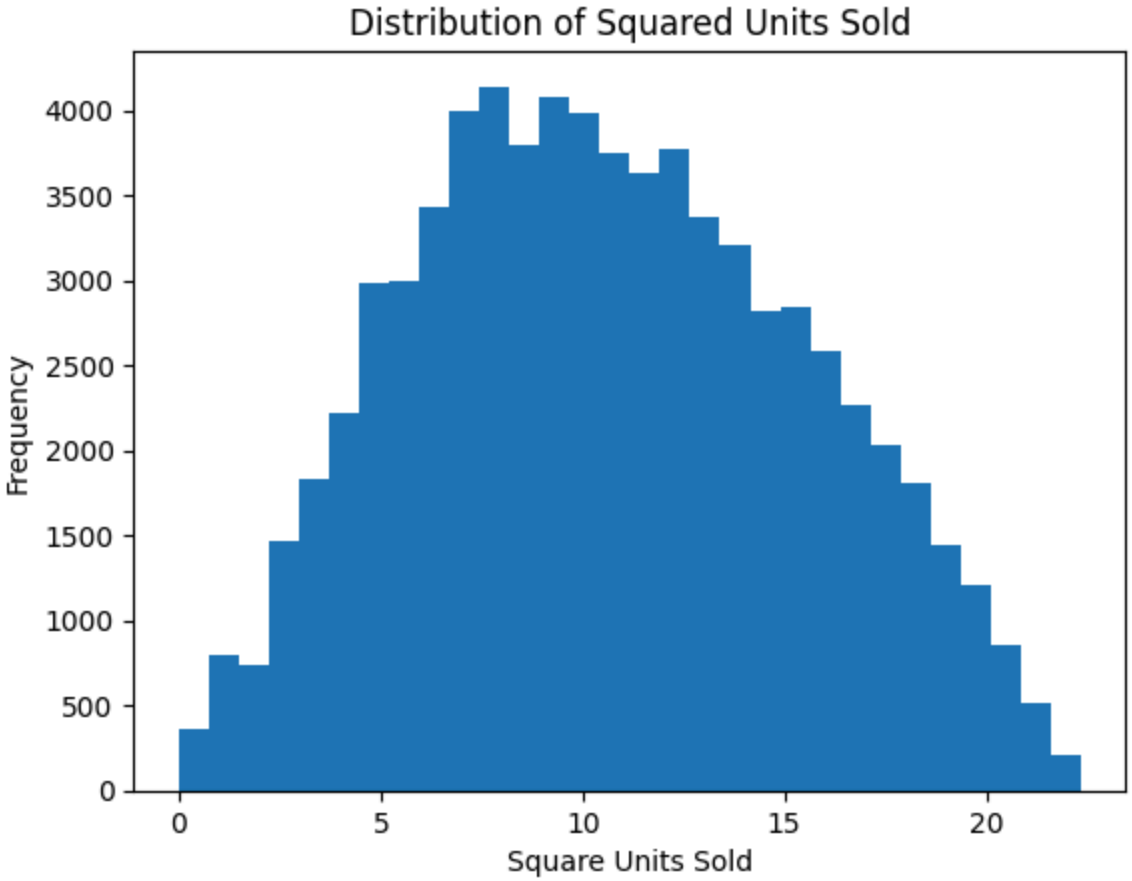


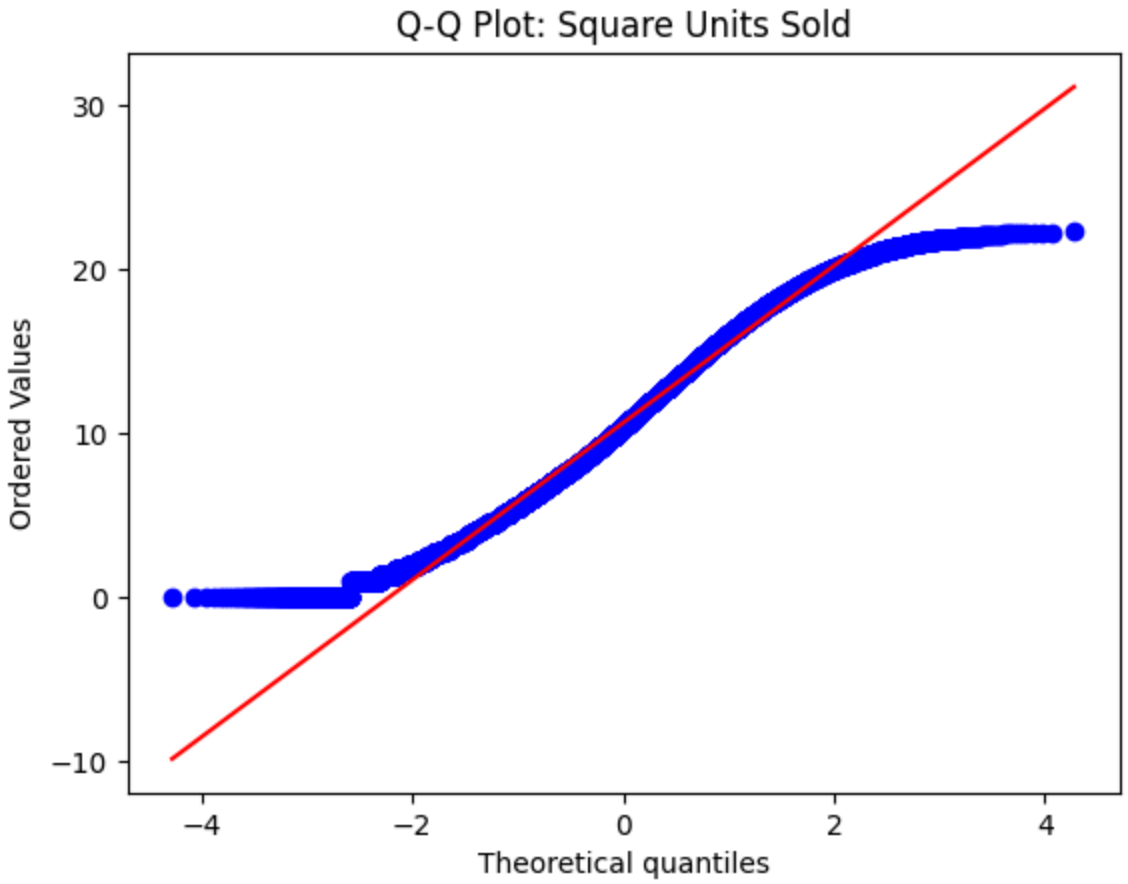



```
Skewness of Log Units Sold: -1.1149201138365896
```

In [298…

```
df['UnitsSold_Sqrt'] = np.sqrt(df['Units Sold'])
ySquared = df['UnitsSold_Sqrt']

plt.hist(ySquared, bins=30)
plt.title("Distribution of Squared Units Sold")
plt.xlabel("Square Units Sold")
plt.ylabel("Frequency")
plt.show()

stats.probplot(ySquared, dist="norm", plot=plt)
plt.title("Q-Q Plot: Square Units Sold")
plt.show()

print("Skewness of Square Units Sold:", ySquared.skew())
```






```
Skewness of Square Units Sold: 0.13521001730548227
```

Start the Stepwise Process - split test/train - using Leave K out because we'll be doing time series.

- A time-based train-test split was used across all models to preserve temporal ordering and prevent data leakage. This ensures model performance reflects real-world forecasting conditions so I want the same

In [299…

```
# Sort the full dataframe first
df = df.sort_values('Date').reset_index(drop=True)

# Recreate log target AFTER sorting
df['UnitsSold_Sqrt'] = np.sqrt(df['Units Sold'])

y_squared = df['UnitsSold_Sqrt']

# Recreate X AFTER sorting
X = df.drop(columns=['Units Sold', 'UnitsSold_Sqrt', 'Date', 'log_units_sold'])

# Convert categorical variables to dummy variables
X = pd.get_dummies(X, drop_first=True)

# Make sure all X values are numeric
```

```
X = X.astype(float)

# Time-based split
split_index = int(len(df) * 0.7)

X_train = X.iloc[:split_index]
X_test  = X.iloc[split_index:]

y_train = y_squared.iloc[:split_index]
y_test  = y_squared.iloc[split_index:]
```

In [300…

```
def stepwise_selection(X, y, threshold_in=0.05, threshold_out=0.10):
    included = []

    while True:
        changed = False

        # Forward step
        excluded = list(set(X.columns) - set(included))
        new_pvals = pd.Series(index=excluded, dtype=float)

        for new_col in excluded:
            model = sm.OLS(y, sm.add_constant(X[included + [new_col]])).fit()
            new_pvals[new_col] = model.pvalues[new_col]

        if not new_pvals.empty:
            best_pval = new_pvals.min()
            if best_pval < threshold_in:
                best_feature = new_pvals.idxmin()
                included.append(best_feature)
                changed = True
                print(f"Add {best_feature} with p-value {best_pval:.6f}")

        # Backward step
        if included:
            model = sm.OLS(y, sm.add_constant(X[included])).fit()
            pvalues = model.pvalues.iloc[1:]
            worst_pval = pvalues.max()

            if worst_pval > threshold_out:
                worst_feature = pvalues.idxmax()
                included.remove(worst_feature)
                changed = True
                print(f"Drop {worst_feature} with p-value {worst_pval:.6f}")

        if not changed:
            break

    return included

selected_features = stepwise_selection(X_train, y_train)

print("\nSelected Features:")
print(selected_features)
```

```
Add Inventory Level with p-value 0.000000

Selected Features:
['Inventory Level']
```

In [301…
```
# Overall this is only suggesting inventory level which is not a multiple linear re
# rather it is a simple linear regression. Going to try using the best subset metho
```

In [302…
```
# Check Best Subset Approach for MLR
```

In [303…
```
import itertools
import pandas as pd
import statsmodels.api as sm
import numpy as np

y = df['Units Sold']

# Potential independent variables
X = df[[
    'Inventory Level',
    'Holiday/Promotion',
    'Units Ordered',
    'Price',
    'Discount',
    'Competitor Pricing'
    ]]

results = []

for k in range(1, len(X.columns) + 1):

    for combo in itertools.combinations(X.columns, k):

        X_subset = X[list(combo)]

        X_subset = sm.add_constant(X_subset)

        model = sm.OLS(y_squared, X_subset).fit()

        results.append({
            'Variables': combo,
            'Adjusted_R2': model.rsquared_adj,
            'AIC': model.aic,
            'BIC': model.bic
        })

results_df = pd.DataFrame(results)

# Sort by Adjusted R²
results_df = results_df.sort_values(
    by='Adjusted_R2',
    ascending=False
)

print(results_df.head(10))
```

```
                                          Variables  Adjusted_R2  \
8                           (Inventory Level, Price)     0.327092
10             (Inventory Level, Competitor Pricing)     0.327092
25           (Inventory Level, Units Ordered, Price)     0.327085
22       (Inventory Level, Holiday/Promotion, Price)     0.327085
27  (Inventory Level, Units Ordered, Competitor Pr...     0.327085
24  (Inventory Level, Holiday/Promotion, Competito...     0.327084
28                (Inventory Level, Price, Discount)     0.327083
29      (Inventory Level, Price, Competitor Pricing)     0.327083
30    (Inventory Level, Discount, Competitor Pricing)     0.327083
41  (Inventory Level, Holiday/Promotion, Units Ord...     0.327078

              AIC            BIC
8   408522.638507  408550.237258
10  408522.681740  408550.280491
25  408524.404583  408561.202917
22  408524.442095  408561.240429
27  408524.448371  408561.246705
24  408524.484689  408561.283024
28  408524.635475  408561.433810
29  408524.637768  408561.436103
30  408524.678629  408561.476963
41  408526.210486  408572.208404
```

In [304…

```
# Choosing Inventory Level and Price as my best multiple linear regression. It has
# the lowest AIC and BIC scores. Also it provides multiple independent variables
# Now fitting the model
```

In [305…

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler

y = df['Units Sold']

X = df[['Inventory Level', 'Price']]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_scaled)

feature_names = poly.get_feature_names_out(X.columns)

X_poly_df = pd.DataFrame(
    X_poly,
    columns=feature_names
)

X_poly_df = sm.add_constant(X_poly_df)

model = sm.OLS(y, X_poly_df).fit()

print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:             Units Sold   R-squared:                       0.348
Model:                            OLS   Adj. R-squared:                  0.348
Method:                 Least Squares   F-statistic:                     7808.
Date:                Sun, 10 May 2026   Prob (F-statistic):               0.00
Time:                        11:25:34   Log-Likelihood:            -4.3097e+05
No. Observations:               73100   AIC:                         8.619e+05
Df Residuals:                   73094   BIC:                         8.620e+05
Df Model:                           5
Covariance Type:            nonrobust
================================================================================
                   coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------------
const          136.8181      0.611    224.079      0.000     135.621     138.015
Inventory Level  64.2703      0.325    197.572      0.000      63.633      64.908
Price            -0.4696      0.325     -1.444      0.149      -1.107       0.168
Inventory Level^2   -0.3516      0.364     -0.967      0.334      -1.065       0.361
Inventory Level Price    -0.5069      0.327     -1.552      0.121      -1.147       0.133
Price^2           0.0030      0.364      0.008      0.994      -0.710       0.716
==============================================================================
Omnibus:                       23.626   Durbin-Watson:                   1.989
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               22.347
Skew:                           0.020   Prob(JB):                     1.40e-05
Kurtosis:                       2.925   Cond. No.                         4.01
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
```

In [306…

```
# Check Hypothesis testing through P-values
```

In [308…

```
print(model.pvalues)

alpha = 0.05

if model.f_pvalue < alpha:
    print("Reject H0: Model is statistically significant")
else:
    print("Fail to reject H0")
```

```
const                   0.000000
Inventory Level         0.000000
Price                   0.148806
Inventory Level^2       0.333739
Inventory Level Price   0.120575
Price^2                 0.993513
dtype: float64
Reject H0: Model is statistically significant
```

In [ ]:
```
# Ananlysis
# Due to the inventory level p value is at 0, this concludes that
# there is a less than 5% probability that the observed results occurred
# by random chance alone. We will reject the null hypothesis.
# Proving this model is statistically significant
```

Check Model Performance

In [254…
```
y = df['Units Sold']

X = X_poly_df

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=16
)

final_model = sm.OLS(y_train, X_train).fit()

y_pred = final_model.predict(X_test)

r2 = r2_score(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

print("Test R-squared:", r2)
print("Test RMSE:", rmse)
```

```
Test R-squared: 0.35419918763852176
Test RMSE: 86.92907436946379
```

Cross Validation

In [255…
```
from sklearn.model_selection import TimeSeriesSplit, cross_val_score
from sklearn.linear_model import LinearRegression

X_train_selected = X_train[selected_features]

tscv = TimeSeriesSplit(n_splits=5)
```

```
lr = LinearRegression()

cv_scores = cross_val_score(
    lr,
    X_train_selected,
    y_train,
    cv=tscv,
    scoring='r2'
)

print("Time Series CV R-squared scores:", cv_scores)
print("Average Time Series CV R-squared:", cv_scores.mean())
```

```
Time Series CV R-squared scores: [0.34176754 0.33918812 0.35164055 0.36553279 0.3341
3664]
Average Time Series CV R-squared: 0.3464531289242969
```

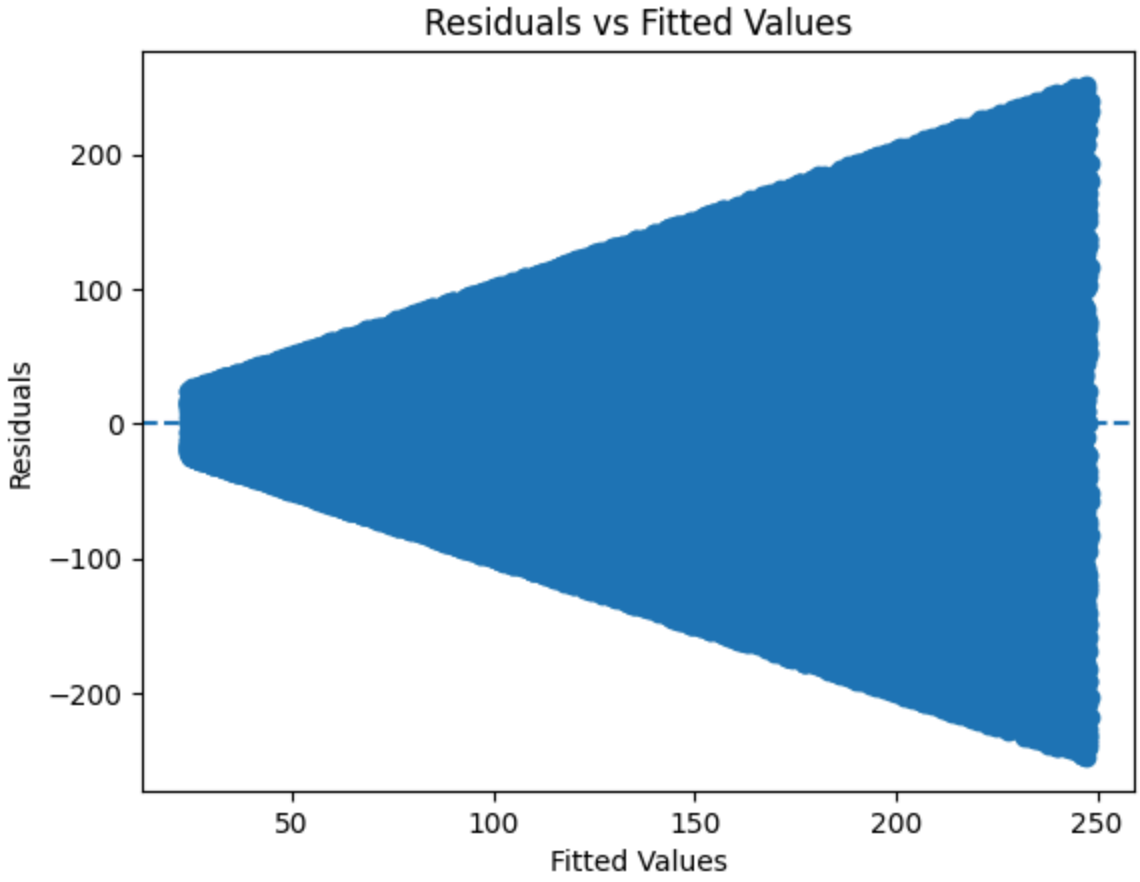
In [256…

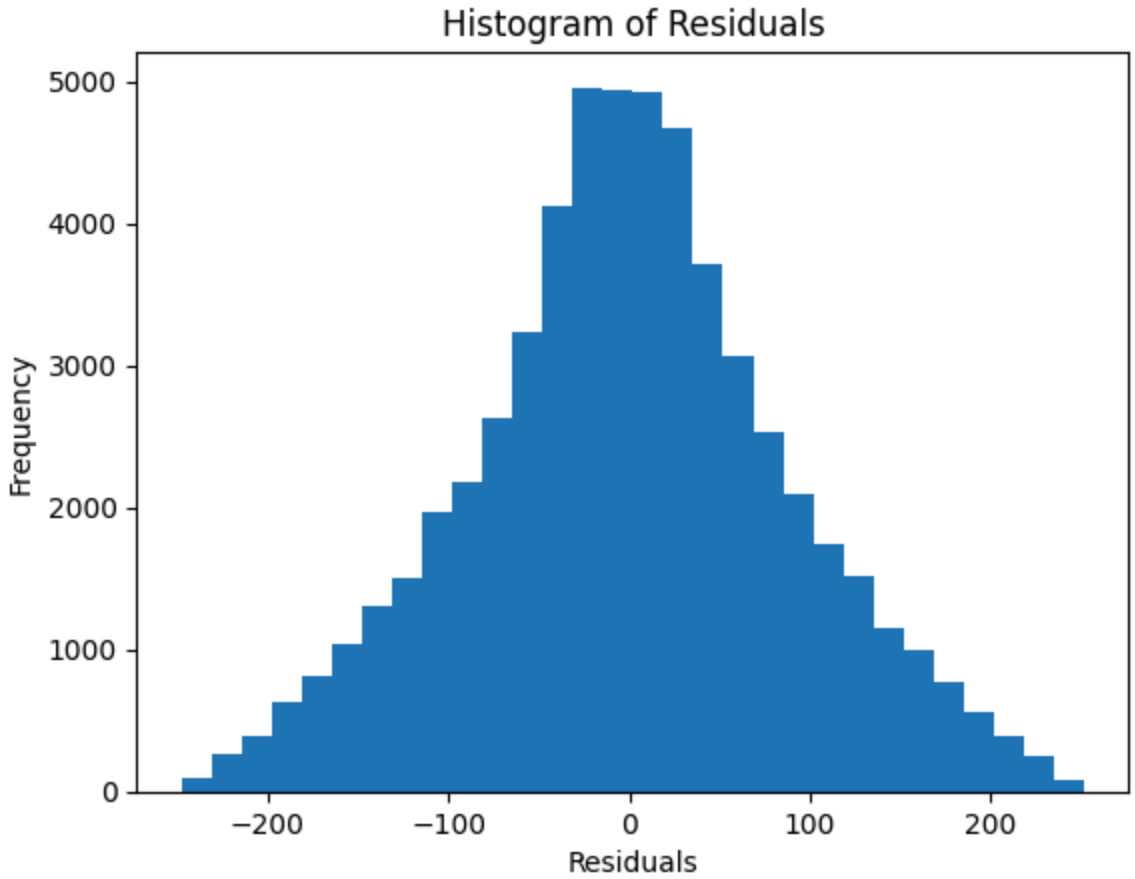
```
# With an average time Series CV R^2 value of 32.5%, this indicates that our model
# have high vairation, and performance is stable
```

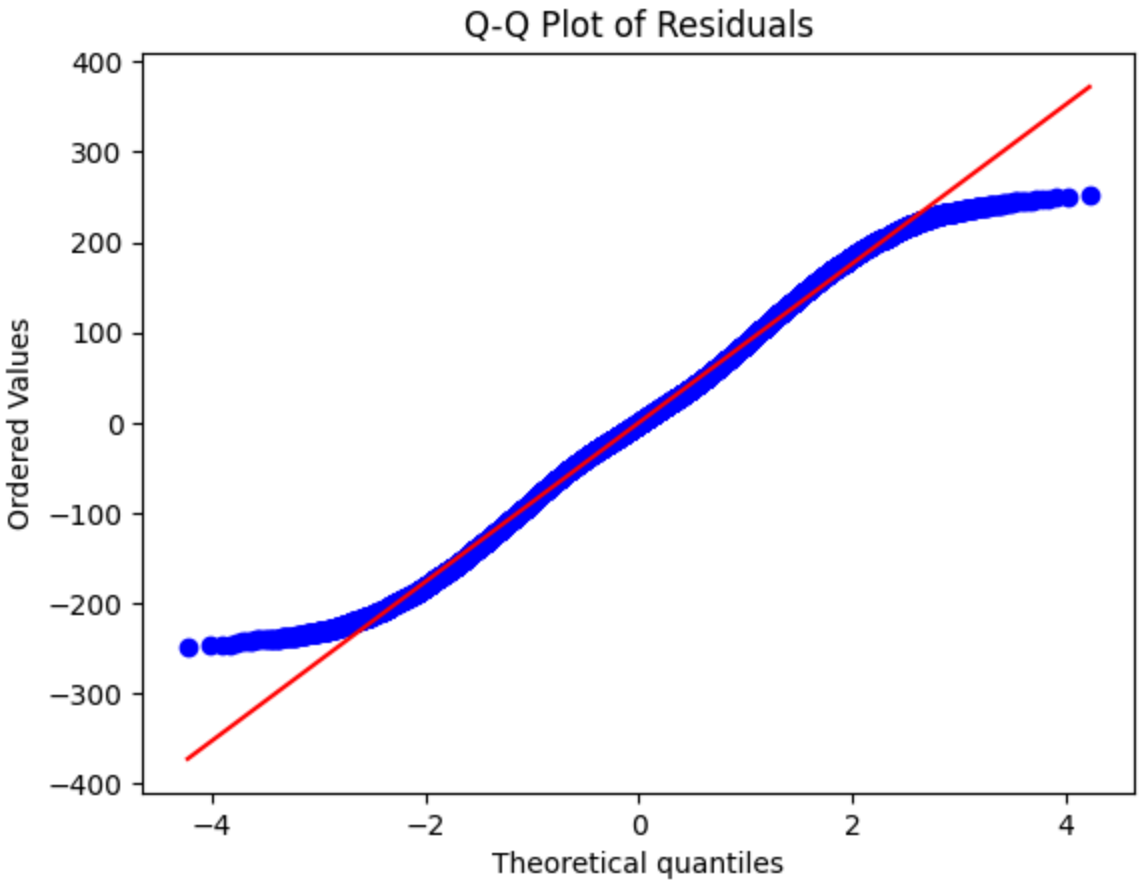
Residual Check

In [257…

```
residuals = final_model.resid
fitted = final_model.fittedvalues

# Residuals vs fitted
plt.scatter(fitted, residuals)
plt.axhline(y=0, linestyle='--')
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted Values")
plt.show()

# Histogram of residuals
plt.hist(residuals, bins=30)
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.title("Histogram of Residuals")
plt.show()

# Q-Q plot of residuals
stats.probplot(residuals, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals")
plt.show()
```


Residuals vs Fitted Values
Residuals
Fitted Values
200
100
0
−100
−200
50
100
150
200
250


Histogram of Residuals
Frequency
Residuals
5000
4000
3000
2000
1000
0
−200
−100
0
100
200



In [63]:
```
from sklearn.model_selection import train_test_split

X = df.drop(columns=['Price'])
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

In [258…
```
# Decision Trees
```

In [275…
```
from sklearn.tree import DecisionTreeClassifier

X = pd.get_dummies(X)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=16)

DT = DecisionTreeClassifier()
DT2 = DecisionTreeClassifier()

DT.fit(X_train,y_train)

DT_Predictions_Train= DT.predict(X_train)
DT_Predictions_Test = DT.predict(X_test)

DT_Predictions_Train
```

Out[275… array([97, 207, 249, ..., 115, 56, 100], shape=(58480,))

In [276…
```
print("Accuracy Rate for the Training Set:", DT.score(X_train, y_train))
print("Accuracy Rate for the Test Set:", DT.score(X_test, y_test))
```

```
Accuracy Rate for the Training Set: 0.9930574555403556
Accuracy Rate for the Test Set: 0.004240766073871409
```

In [ ]:
```
# In this training set, 99.3% of records have been classified
# correctly. However the test set had 0.4% which indicates new
# data is not working. This could mean our decision tree is
# overfitted.
```

In [265…
```
# Random Forest
```

In [267…
```
from sklearn.ensemble import RandomForestClassifier

RFC = RandomForestClassifier(
    n_estimators=50,
    max_depth=15,
    n_jobs=-1
)

RFC.fit(X_train, y_train)
```

Out[267… ▾ RandomForestClassifier ⓘ ?

▸ Parameters

In [268…
```
RFC_Predictions_Train= RFC.predict(X_train)
RFC_Predictions_Test = RFC.predict(X_test)
```

In [269…
```
RFC_Predictions_Train
```

Out[269… array([97, 61, 66, ..., 115, 56, 433], shape=(58480,))

In [270…
```
print("Accuracy Rate for the Training Set:", RFC.score(X_train, y_train))
print("Accuracy Rate for the Test Set:", RFC.score(X_test, y_test))
```

```
Accuracy Rate for the Training Set: 0.5734097127222982
Accuracy Rate for the Test Set: 0.004445964432284542
```

In [ ]:
```
# In this training set, 57.3% of records have been classified
# correctly. However the test set had 0.4% which indicates new
# data is not working. This could mean our decision tree is
# overfitted.
```